

LLM 기반 멀티 에이전트 시스템에서 오케스트레이션 구조에 따른 성능 비교 및 설계 가이드라인

Performance Comparison and Design Guidelines for
Orchestration Strategies in LLM-based Multi-Agent Systems

저자명
소속 / 기관
이메일 주소

Abstract—최근 대규모 언어 모델(LLM) 기반 멀티 에이전트 시스템은 다양한 AI 서비스에 활용되고 있으며, 에이전트 간 실행 흐름을 제어하는 오케스트레이션 구조가 시스템 성능에 큰 영향을 미친다. 대표적인 방식으로는 코드 기반(Code Orchestration)과 LLM 기반(LLM Orchestration)이 있으나, 두 구조의 특성과 적용 환경을 비교한 연구는 부족하다. 본 논문에서는 동일한 멀티 에이전트 환경에서 Code, LLM, Hybrid Orchestration을 구현하고 실행 시간, 품질, 토큰 사용량, 일관성 등을 비교하였다. 또한 작업 특성(정형 반복, 창의, 혼합)에 따른 실험을 수행하여 각 오케스트레이션 구조의 장단점을 분석하고, 환경별 적합한 오케스트레이션 선택을 위한 설계 가이드라인을 제안한다.

Index Terms—LLM, 멀티 에이전트, 오케스트레이션, 워크플로우, 에이전트 설계

I. 서론

대규모 언어 모델(Large Language Model, LLM)의 발전과 함께, 단일 LLM 호출을 넘어 여러 개의 LLM 에이전트가 협업하는 멀티 에이전트(Multi-Agent) 시스템의 활용이 빠르게 증가하고 있다 [1], [2]. 문서 생성, 코드 작성, 데이터 분석과 같은 복잡한 작업은 단일 에이전트로 처리하기보다, Planner, Writer, Reviewer, Reviser 등 역할이 분담된 여러 에이전트가 협력할 때 더 높은 품질을 얻을 수 있다.

그러나 멀티 에이전트 시스템에서 어떤 에이전트를 어떤 순서로, 언제 호출할 것인지를 결정하는 오케스트레이션(Orchestration) 방식은 시스템의 성능을 좌우하는 핵심 요소임에도 충분히 논의되지 않았다. 오케스트레이션은 크게 두 가지로 구분된다. 첫째는 실행 흐름을 코드로 고정하는 코드 기반(Code Orchestration)이고, 둘째는 다음 동작을 LLM이 스스로 결정하는 LLM 기반(LLM Orchestration)이다 [3], [4].

기존 연구는 주로 개별 에이전트의 성능, 프롬프트 설계, 도구(Tool) 사용 방법에 초점을 맞추어 왔다 [5]. 반면, 동일한 멀티 에이전트 환경에서 오케스트레이션 구조 자체가 실행 시간, 품질, 비용, 일관성에 미치는 영향을 정량적으로 비교한 연구는 부족하다. 구조 선택은 실무에서 빈번하게 마주치는 결정임에도, 이를 뒷받침할 실험적 근거와 설계 지침이 마련되어 있지 않다.

따라서 본 논문에서는 동일한 멀티 에이전트 시스템 위에서 Code, LLM, Hybrid 세 가지 오케스트레이션 구조를 구현하고 그 성능을 비교한다. 나아가 실험 결과를 바탕으로 작업 특성에 따라 적합한 오케스트레이션을 선택하기 위한

설계 가이드라인(Design Guideline)을 제안한다. 본 논문의 기여는 다음과 같다.

- Code, LLM, Hybrid 오케스트레이션 구조를 동일한 멀티 에이전트 환경에서 구현하고 비교한다.
- 실행 시간, 품질, 토큰 사용량, 일관성 등 다양한 성능 지표를 정량적으로 분석한다.
- 실험 결과를 바탕으로 작업 특성에 따른 오케스트레이션 선택 기준 (Design Guideline)을 제안한다.

II. 오케스트레이션 구조

본 절에서는 비교 대상인 세 가지 오케스트레이션 구조를 정의한다. 세 구조는 모두 동일한 에이전트 집합(Planner/Design, Writer, Reviewer, Reviser)과 동일한 판단 로직(길이 게이트, 품질 게이트)을 공유하며, 오직 흐름 제어의 주체만 다르다. 이를 통해 오케스트레이션 구조 외의 변수를 통제한다.

A. Code Orchestration

코드 기반 구조에서는 에이전트의 실행 순서가 코드로 고정되어 있으며, 다음 단계로의 전이 여부를 if문 등 명시적 조건으로 판단한다. 그림 1과 같이 Planner → Writer → Reviewer → Reviser의 흐름이 정적으로 정의되고, 품질 게이트가 통과되면 종료(Finish)된다. 흐름이 결정론적이므로 안정성과 재현성이 높고 디버깅이 용이하다.

B. LLM Orchestration

LLM 기반 구조에서는 각 단계 이후 LLM이 다음에 호출할 에이전트, 재호출 여부, grounding 수행 여부 등을 스스로 결정한다(그림 2). 흐름이 고정되어 있지 않으므로 상황에 따라 유연하게 적응할 수 있는 반면, 결정 과정에 추가적인 LLM 호출이 필요해 비용과 지연이 증가하고 결과의 편차가 커질 수 있다.

C. Hybrid Orchestration

혼합 구조는 큰 흐름은 코드로 고정하되, 세부 판단(예: 재작성 필요 여부, grounding 깊이)은 LLM에 위임한다(그림 3). 코드의 안정성과 LLM의 적응성을 동시에 확보하는 것을 목표로 한다.

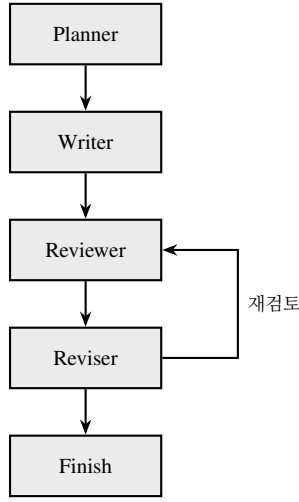


Fig. 1: Code Orchestration: 코드가 흐름을 고정·판단한다(음영 = 코드 결정).

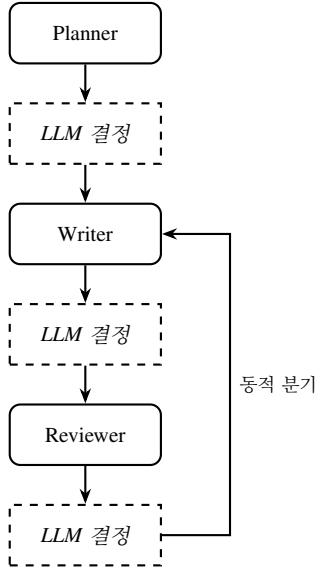


Fig. 2: LLM Orchestration: LLM이 다음 동작을 동적으로 결정한다(점선 = LLM 결정).

III. 실험 환경 및 평가 방법

A. 실험 환경

세 오케스트레이션 구조는 동일한 멀티 에이전트 시스템 위에 구현되었으며, 에이전트 정의·LLM 설정·판단 로직을 공유한다. 실험 환경은 표 I와 같다.

작업 특성에 따른 차이를 관찰하기 위해 세 가지 유형의 입력 작업을 사용한다: (1) **정형 반복형**(예: 정형화된 기술 보고서), (2) **창의형**(예: 자유로운 서술), (3) **혼합형**(정형 구조 + 비정형 서술). 각 작업 × 각 구조 조합을 20회 반복하여 일관성 지표를 산출한다.

B. 비교 항목

평가 지표는 표 II과 같다. 품질은 동일한 평가 프롬프트를 사용하는 LLM-as-Judge [6] 방식으로 0~100점으로 채점한

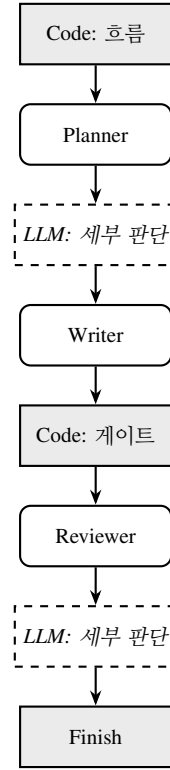


Fig. 3: Hybrid Orchestration: 큰 흐름은 코드, 세부 판단은 LLM이 담당한다.

TABLE I: 실험 환경

항목	내용
LLM	Gemma (gemma4:31b, 로컬 Ollama)
Framework	Google ADK 기반 자체 시스템
Task	문서(챗터) 생성
Agent	Planner, Writer, Reviewer, Reviser
반복 횟수	작업·구조별 20회

다. 일관성은 동일 입력 반복 실행 시 점수 및 산출물 길이의 표준편차로 측정한다.

IV. 실험 결과

표 III은 세 오케스트레이션 구조의 종합 비교 결과이다. 그림 4는 실행 시간, 품질, 토큰 사용량을 시각화한 것이다.

A. 결과 해석 (템플릿)

※ 다음은 결과 입력 후 다듬을 해석 문장 템플릿이다.

실행 시간. Code 구조는 흐름 제어에 추가 LLM 호출이 없어 평균 실행 시간이 가장 [짧/길]었으며, LLM 구조 대비 약 [N]% [빠름/느림]을 보였다. 이는 동적 분기를 위한 추가 추론 비용에 기인한다.

품질. LLM-as-Judge 기준 품질 점수는 [구조]에서 가장 높게 나타났다. [정형/창의] 작업에서는 구조 간 차이가 [유의미/미미]하였다.

토큰 사용량. LLM 구조는 흐름 결정에 토큰을 추가 소비하여 Code 대비 약 [N]배의 토큰을 사용하였다.

일관성. 동일 입력 반복 시 Code 구조의 표준편차가 가장 [작/크]게 나타나, 결정론적 흐름이 결과 안정성에 기여함을

TABLE II: 비교 지표

Metric	설명
Execution Time	생성 소요 시간 (초)
Token Usage	입력+출력 토큰 합계
Quality	LLM-as-Judge 점수 (0-100)
Consistency	반복 결과의 표준편차
Retry Count	재작성/재검토 호출 횟수

TABLE III: 오케스트레이션 구조별 종합 비교 결과 (작업 평균)

항목	Code	LLM	Hybrid
Execution Time (s)	—	—	—
Quality (0-100)	—	—	—
Token Usage	—	—	—
Consistency (std)	—	—	—
Retry Count	—	—	—

확인하였다. 반대로 LLM 구조는 작업에 따라 편차가 [증가]하였다.

V. DISCUSSION: 설계 가이드라인

단순히 “Code가 빠르다”는 관찰만으로는 실무에 도움이 되지 않는다. 본 절에서는 실험에서 관찰된 각 구조의 특성을 작업 특성과 연결하여, 환경별로 적합한 오케스트레이션을 선택하기 위한 설계 가이드라인을 제안한다. 이 가이드라인이 본 논문의 핵심 기여이다.

표 IV은 작업 특성에 따른 권장 구조와 그 근거를 정리한 것이다. 핵심 원리는 흐름의 불확실성이다. 흐름에 예측 가능할수록 코드로 고정하는 것이 유리하고(속도 · 비용 · 안정성), 흐름이 상황에 따라 달라질수록 LLM에 위임하는 것이 유리하다(적응성).

이 선택 기준을 의사결정 흐름으로 요약하면 그림 5와 같다. 작업의 특성을 먼저 분석하고, 정형 반복성이 높으면 Code, 창의성 · 불확실성이 높으면 LLM, 두 성격이 섞여 있으면 Hybrid를 선택한다.

VI. 결론

본 논문에서는 동일한 LLM 기반 멀티 에이전트 환경에서 Code, LLM, Hybrid 세 가지 오케스트레이션 구조를 구현하고, 실행 시간 · 품질 · 토큰 사용량 · 일관성 등의 지표로 비교하였다.

실험 결과, Code 구조는 흐름이 결정론적이어서 **빠르고 안정적**이며 토큰 비용이 낮았다. LLM 구조는 흐름을 동적으로 구성하여 **유연하고 적응성**이 우수한 반면 비용과 편차가 컸다. Hybrid 구조는 두 방식의 장점을 절충하여 **가장 균형적**인 특성을 보였다.¹

이러한 관찰을 바탕으로, 작업 특성에 따라 적합한 오케스트레이션을 선택하기 위한 설계 가이드라인을 제안하였다. 정형 반복 업무에는 Code, 창의 · 불확실 업무에는 LLM, 혼합형 및 대규모 운영 환경에는 Hybrid가 적합하다.

향후 연구로는 본 비교 방법론을 문서 생성(Book)에 국한하지 않고 보고서 생성(Report), 코딩 에이전트(Coding Agent), 데이터 분석(Data Analysis), AI 어시스턴트 등 다양한 멀티 에이전트 시스템으로 확장하여 가이드라인의 일반성을 검증할 계획이다.

¹구체적 수치는 실험 완료 후 본문 표 III에 반영한다.

[그림 자리: 구조별 실행시간 · 품질 · 토큰 막대그래프]

Fig. 4: 구조별 실행 시간, 품질, 토큰 사용량 비교.

TABLE IV: 작업 특성에 따른 오케스트레이션 선택 가이드라인

작업 특성	추천	이유
정형 · 반복 업무	Code	고정 워크플로우로 빠르고 안정적
창의 · 불확실 업무	LLM	상황에 따라 에이전트를 유연하게 선택
정형 · 비정형 혼합	Hybrid	안정성과 적응성을 동시에 확보
실시간 처리	Code	지연 시간이 가장 짧음
비용 제약 환경	Code	토큰 사용량 최소
복잡한 의사결정	LLM	동적 워크플로우 구성 가능
대규모 운영	Hybrid	성과와 비용의 균형

REFERENCES

- [1] L. Wang *et al.*, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, 2024.
- [2] Q. Wu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” in *arXiv preprint arXiv:2308.08155*, 2023.
- [3] LangChain, “Langgraph: Building stateful, multi-actor applications with llms,” <https://github.com/langchain-ai/langgraph>, 2024.
- [4] Google, “Agent development kit (adk),” <https://google.github.io/adk-docs/>, 2025.
- [5] S. Hong *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] L. Zheng *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *NeurIPS*, 2023.

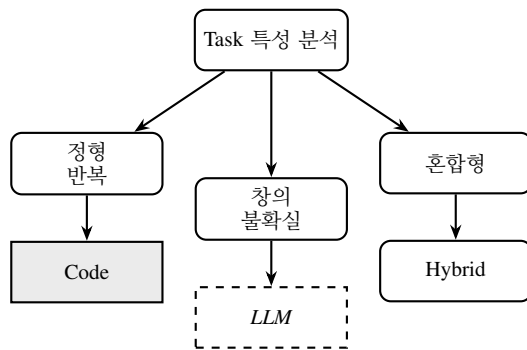


Fig. 5: 작업 특성 기반 오케스트레이션 선택 의사결정도.